Academia de Studii Economice din București

Facultatea de Cibernetică, Statistică și Informatică Economică

Master Informatica Economica

Documentatie proiect

Afaceri electronice

Student GONGĂNĂU VALENTIN-MARIAN

An studiu 2, sem. 1

2026

AE Hardware – documentatie

Magazin online de componente PC, construit pe **PrestaShop 8**, cu trei functionalitati proprii:

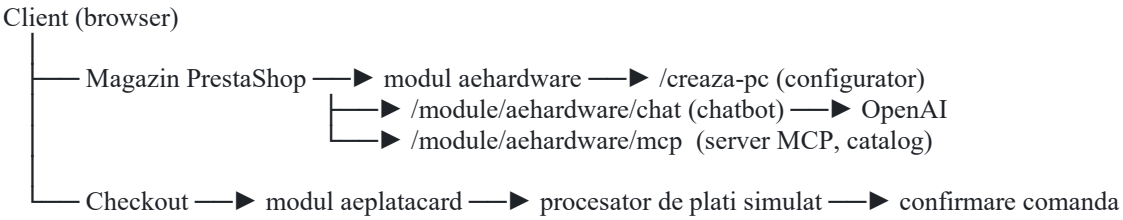
1. **Asistent AI** (chatbot) care recomanda produse si configuratii din catalogul real al magazinului.
2. **Configurator „Creaza propriul tau PC”** (custom build), care arata doar componente compatibile intre ele.
3. **Plata online cu cardul**, printr-un procesator de plati simulat, cu date semnate.

Site live	https://produse-software-ce--afaceri-electronice.gonganau.ro
Administrare	https://produse-software-ce--afaceri-electronice.gonganau.ro/admin-ae
Cod sursa	https://github.com/lucrari-ase-valentin-gonganau/produse-software-ce--afaceri-electronice

---

1. Arhitectura pe scurt

- **PrestaShop 8 (Apache) + MariaDB 10.11**, pornite cu Docker Compose.
- **Doua module proprii**, in modules/:
  - aehardware – configuratorul PC, serverul MCP al catalogului si chatbotul AI;
  - aeplatacard – metoda de plata „Card bancar (online)”.
- **Catalogul** (categorii, produse, caracteristici tehnice, imagini) e generat automat de build_seed.py si desene_hardware.py la construirea imaginii Docker, apoi importat de docker/sync.php la fiecare pornire a containerului.



---

2. Asistent AI (chatbot)

Bulina de chat disponibila pe toate paginile magazinului. Clientul scrie in limbaj natural (ex. *„vreau un PC pentru Cyberpunk 2077, buget 6000 lei”*) si primeste recomandari **doar cu produse existente in magazin**, cu link, pret si stoc.

Cum functioneaza

- Modelul AI este de la **OpenAI** (implicit gpt-5-mini), apelat prin Chat Completions cu *function calling*.

- Datele despre produse nu sunt inventate de model: el apeleaza **uneltele serverului MCP** al magazinului (Model Context Protocol), care citesc catalogul real:

Unealta MCP	Ce face
lista_categorii	categoriile magazinului
cauta_produse	cautare dupa text, categorie, pret
detalii_produs	specificatiile complete ale unui produs
produse_compatibile	ce componente se potrivesc cu cele deja alese
completeaza_configuratie	completeaza o configuratie PC compatibila, in limita unui buget
verifica_configuratie	verifica compatibilitatea si consumul unei configuratii

- Unelte suplimentare, doar in chatbot:
  - cerinte_joc – cauta pe internet cerintele de sistem ale unui joc (model OpenAI cu web search);
  - adauga_in_cos – pune produsele recomandate direct in cosul clientului, doar la cererea lui.
- Cand raspunsul contine o configuratie completa, aceasta apare ca **card** in chat (componente, total, buton „Adauga in cos”), cu datele exacte din unealta, nu rescrise de model.
- **Limita de mesaje** pe ora / IP (implicit 20), ca sa nu fie consumata cheia API.
- Serverul MCP e public (/module/aehardware/mcp, doar citire), deci poate fi folosit si de alti clienti MCP (ex. Claude, alte aplicatii AI).

Configurare: Admin › Module › AE Hardware › Configureaza – cheia OpenAI, modelul, modelul de cautare, limita de mesaje si buton de test. Setarile pot veni si din variabile de mediu (AE_CHAT_API_KEY, AE_CHAT_MODEL, AE_CHAT_API_URL, AE_CHAT_SEARCH_MODEL).

---

3. Configurator „Creaza propriul tau PC” (custom build)

Pagina **/creaza-pc**, un ghid pas cu pas prin cele 7 componente ale unui PC:

Procesor → Placa de baza → Memorie RAM → Placa video → SSD → Sursa → Carcasa

- La fiecare pas se afiseaza **doar produsele compatibile** cu ce s-a ales deja; pentru fiecare pas se explica si conditia (ex. *„socket AM5, ca placa de baza”*).
- **Reguli de compatibilitate** verificate:
  - socketul procesorului = socketul placii de baza;
  - tipul memoriei (DDR4 / DDR5) suportat de procesor si de placa de baza;
  - formatul placii de baza (ATX, mATX, ITX) incape in carcasa;

    - **sursa** are putere suficienta: consumul estimat al componentelor + 25 W (cooler, USB) + **30% rezerva**. Functioneaza si invers: daca sursa e aleasa prima, raman doar componentele care incap in puterea ei.
- Filtre pe fiecare pas, pret total actualizat pe loc, consum pe componente.
- Ultimul pas: rezumatul configuratiei, cu posibilitatea de a reveni la orice pas.
- Butonul **„Adauga configuratia in cos”** pune toate componentele in cosul standard PrestaShop.
- Pe pagina fiecarui produs-componenta apare un link catre configurator.

Aceeasi logica de compatibilitate (AeCompatibility) e folosita si de chatbot, prin serverul MCP, deci configuratiile recomandate de AI sunt mereu valide.

---

4. Plata online cu cardul

Modulul aeplatacard adauga in checkout metoda **„Plata online cu cardul”**. Procesatorul de plati este **simulat**, dar fluxul este acelasi ca la un procesator real (Netopia, Stripe etc.):

1. Clientul alege plata cu cardul si plaseaza comanda → comanda primeste starea **„Se asteapta plata cu cardul”**.
2. Magazinul il trimite pe pagina procesatorului, cu detaliile comenzii (comerciant, comanda, suma, client) **semnate HMAC-SHA256**, ca sa nu poata fi modificate pe drum.
3. Pe pagina procesatorului clientul apasa **„Platit”** sau **„Neplatit”**; procesatorul semneaza rezultatul si il trimite inapoi in magazin.
4. Magazinul verifica semnatura, referinta si suma, apoi:
    - **Platit** → comanda devine **„Plata acceptata”**, cu id-ul tranzactiei (SIM-...), iar clientul ajunge pe pagina „Comanda confirmata”;
    - **Neplatit** → comanda devine **„Eroare la plata”**; clientul poate **reincerca plata** din pagina de eroare sau din istoricul comenzilor.

Cheia de semnare se genereaza automat la instalarea modulului. Statusul platii si id-ul tranzactiei apar si in detaliile comenzii (cont client si admin).

---

5. Rulare si deploy

```
# local: http://localhost:8080 , admin: http://localhost:8080/admin-ae
docker compose up -d --build
```

- La prima pornire PrestaShop se instaleaza automat (limba romana, tara Romania), apoi docker/10-sync.sh importa catalogul si instaleaza / actualizeaza modulele proprii.
- Productie: docker-compose.prod.yml + variabilele din .env.prod, publicare cu deploy.sh.
